SLU Football Tackle Analytics

An In-Depth Look into Level 3 vs. Level 2 Tackling Efficiency

By: Joshua Larson

October 23, 2025

What We Will Cover…

- Grading Scheme
- Data Collection Process
- Data Assembly
- Manipulations
- Visualizations
- Conclusion

Grading Scheme

- Background: Tackling is the backbone of football. In its early days, the game closely mimicked rugby: running, blocking, and tackling the ball carrier. Over the years, tackling has progressed from a violent, out-of-control collision to a process that involves speed, positioning, and technique, all while trying to maintain that violent nature while keeping yourself safe. In today's age, an efficient tackler must be able to TRACK, PREPARE, CONNECT, ACCELERATE, and FINISH.

Track: Evaluates angle, leverage, and speed in closing space

Prepare: Focus on base, pad level, head positioning, and footwork before contact

Connect: Eyes up and winning with point-on-point contact

Accelerate: Maintaining momentum by running feet and hips through

Finish: The control in a wrap, roll, or through motion all the way to the ground

The Other Variables

Base Columns: PLAY #, ODK, DN, DIST, YARD LN, HASH, GN/LS, PERSONNEL, PLAY TYPE, PLAY RESULT, DEF. PLAY CALL

Tackle Columns: TACKLER (Jersey #), TACKLE POS (DL, DE, LB, etc.), TACKLE ENTRY (Front, Side, Rear), LEVEL (3, 2, 1), ENGAGED, MAKE/MISS, YAFC (first chance/contact, ASSISTED, FIELD_TYPE (Box, Alley, Open, Sideline)

The Data

- How is the data collected?
  - Hudl Sideline → Hudl
  - Cutup and Tag
  - In-depth watch on every play with a tackle
  - Only grade first chance tackle attempts
  - Collected leveraging data simultaneously
  - Export to .xlsx

Loading and Standardizing the Tackle Dataset

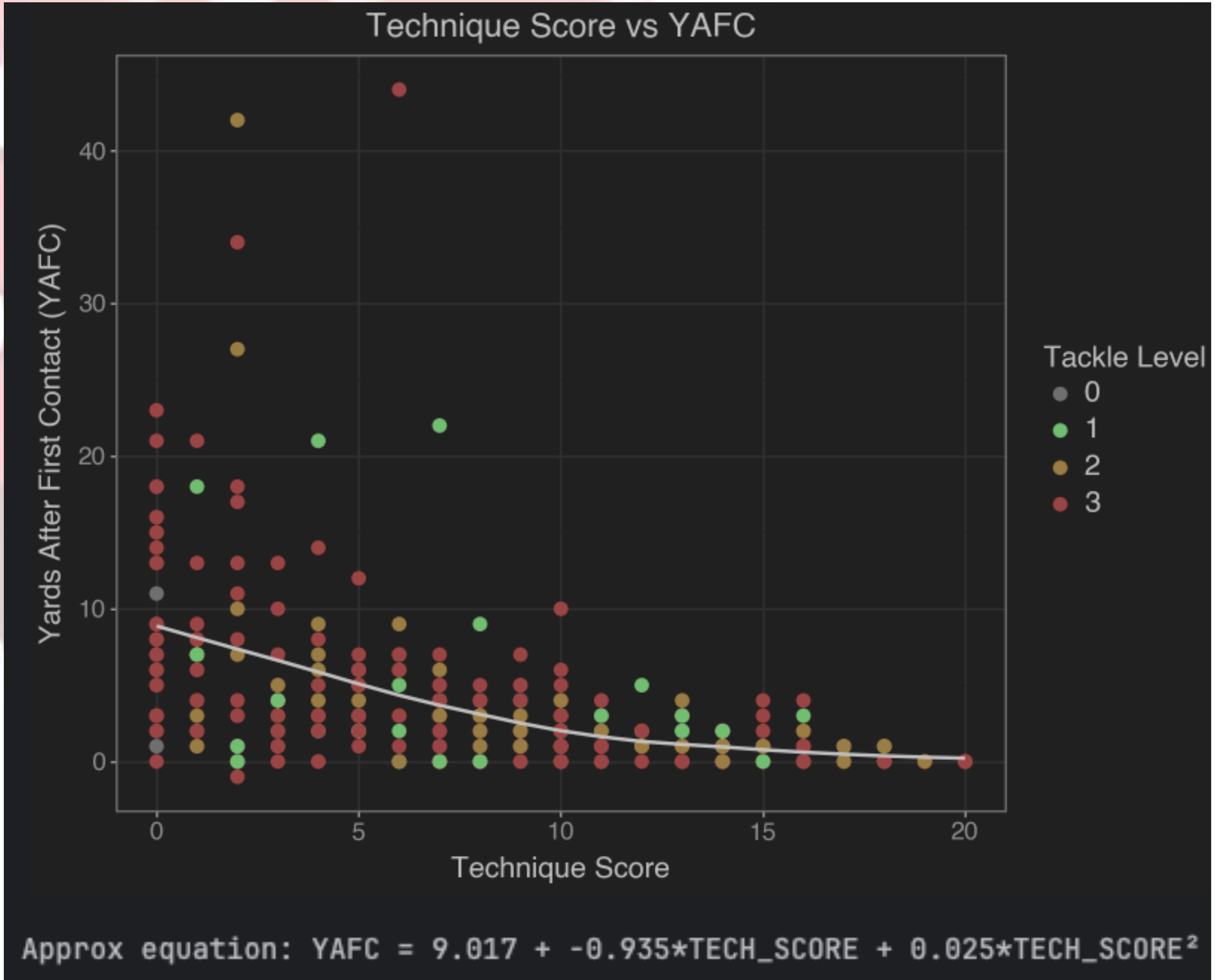
```python
import pandas as pd

# 1) Load
utica = pd.read_excel("../Python_data/UticaTackleData_2025-08-23.xlsx")
moville = pd.read_excel("../Python_data/MorrisvilleTackleData_2025-08-30.xlsx")
norwich = pd.read_excel('../Python_data/NorwichTackleData_2025-09-11.xlsx')
sjf = pd.read_excel('../Python_data/SJFTackleData_2025-09-17.xlsx')
alfred = pd.read_excel('../Python_data/AlfredTackleData_2025-09-20.xlsx')
union = pd.read_excel('../Python_data/UnionTackleData_2025-09-27.xlsx')
hilbert = pd.read_excel('../Python_data/HilbertTackleData_2025-10-04.xlsx')
ithaca = pd.read_excel('../Python_data/IthacaTackleData_2025-10-11.xlsx')

# 2) Tag game and date
utica["GAME"],   utica["GAME_DATE"], utica['WEEK']   = "UT", "2025-08-23", "-1"
moville["GAME"], moville["GAME_DATE"], moville['WEEK'] = "MO", "2025-08-30", '0'
norwich["GAME"], norwich["GAME_DATE"], norwich['WEEK'] = "NR", "2025-09-05", '1'
sjf["GAME"], sjf["GAME_DATE"], sjf['WEEK'] = "SJF", "2025-09-13", '2'
alfred["GAME"], alfred["GAME_DATE"], alfred['WEEK'] = "ALF", "2025-09-20", '3'
union["GAME"], union["GAME_DATE"], union['WEEK'] = "UC", "2025-09-27", '4'
hilbert["GAME"], hilbert["GAME_DATE"], hilbert['WEEK'] = "HC", "2025-10-04", '5'
ithaca["GAME"], ithaca["GAME_DATE"], (ithaca['WEEK']) = "IC", "2025-10-11", '6'

# 3) Combine
tackle = pd.concat([utica, moville, norwich, sjf, alfred, union, hilbert, ithaca], ignore_index=True)

tech_cols = ["TRACK", "PREPARE", "CONNECT", "ACCELERATE", "FINISH"]
tackle["TECH_SCORE"] = tackle[tech_cols].sum(axis=1)
tackle
```

✓ [1] 193ms

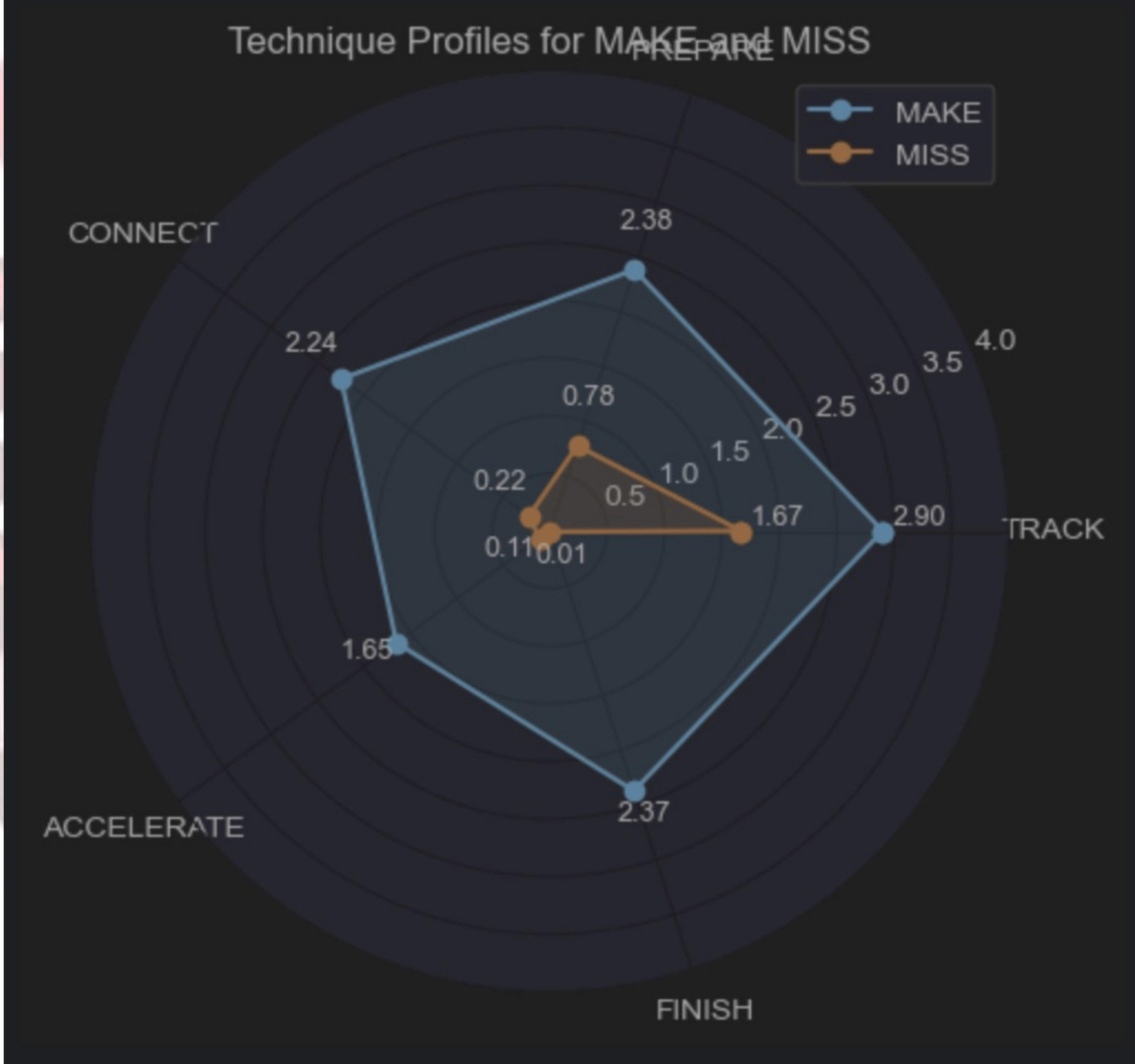
- Packages used:
  - ➢ Pandas
  - ➢ Matplotlib
  - ➢ Numpy
  - ➢ Plotnine
  - ➢ Scipy

Visualizations

- Fit a 2nd-order Polynomial
- Has a quadratic shape; any others would overfit or be too noisy
- Manually coded because Plotnine doesn't supply fitted equations.



Visualizations



Base/Comprehensive Analyses

- Initiated tackle = tackle_clean for all manipulations: 371 → 364
  - Explanation: Several large plays with YAFC>20 were on PENALTIES

```
tackle_clean = tackle.loc[tackle['PLAY RESULT'] != 'PENALTY']
tackle = tackle_clean
```

LEVEL	count
3	229
2	90
1	42
0	3

- 0 denotes that an attempt was either so poor or no effort was given to make an accurate assumption on where they would position their body.

Base/Comprehensive Analyses

LEVEL	MAKE/MISS	proportion
0	MISS	1.000000
1	MAKE	0.714286
	MISS	0.285714
2	MAKE	0.700000
	MISS	0.300000
3	MAKE	0.593886
	MISS	0.406114

Base/Comprehensive Analyses

LEVEL	MAKE/MISS	YAFC
0	MISS	5.000000
1	MAKE	1.833333
	MISS	7.666667
2	MAKE	1.396825
	MISS	7.000000
3	MAKE	1.713235
	MISS	7.473118

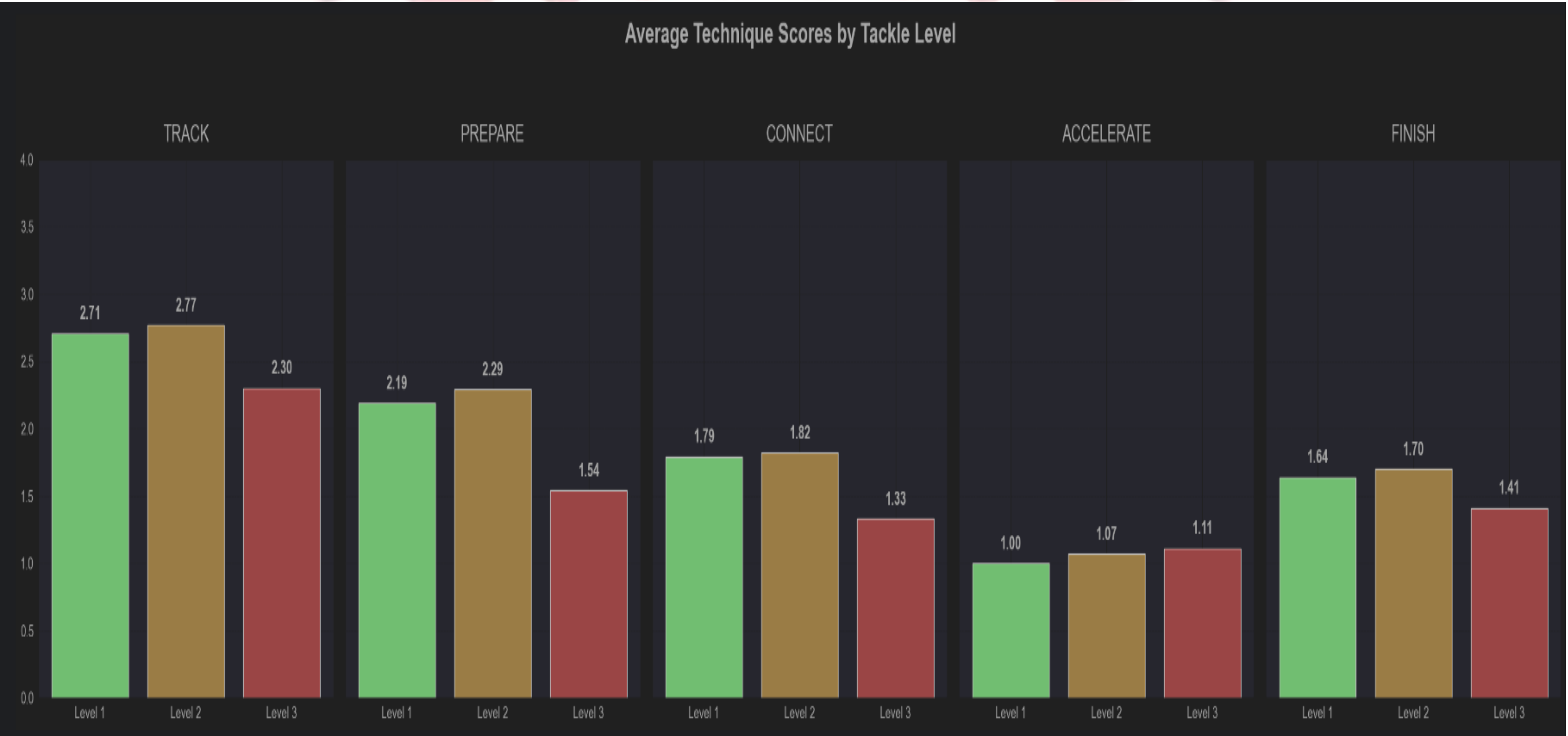
Base/Comprehensive Analyses

```
tackle.groupby("LEVEL")["YAFC"].agg(["min", "max", "mean"])
✓ [15] < 10 ms
```

4 rows ⌄ 4 rows × 3 cols

LEVEL	min	max	mean
0	1	11	5.000000
1	0	22	3.500000
2	0	42	3.077778
3	-1	44	4.052402

Visualizations



Conclusion…